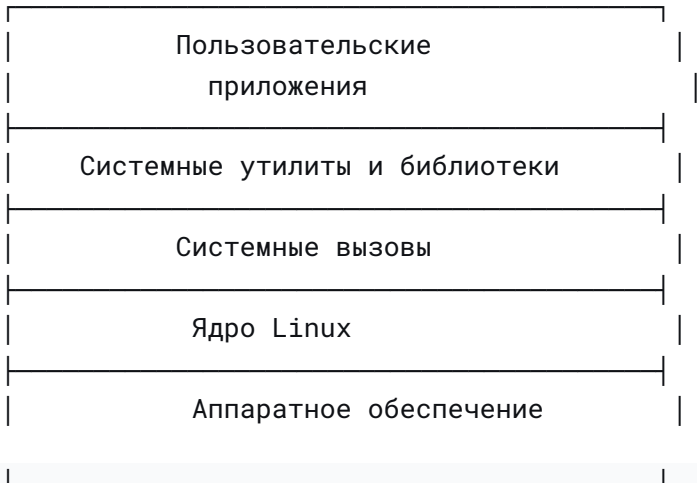


Подробное руководство по архитектуре Linux/Ubuntu

🏗️ Обзор архитектурных уровней

Многоуровневая архитектура Linux



⚙️ Уровень 1: Аппаратное обеспечение (Hardware Layer)

Поддерживаемые архитектуры процессоров

x86/x86-64 (Intel/AMD):

```

bash

```

```
# Проверка архитектуры
uname -m
# x86_64 - 64-битная архитектура
# i686, i386 - 32-битная архитектура
```

```
# Подробная информация о CPU
lscpu
```

```
cat /proc/cpuinfo
```

ARM архитектуры:

```
bash
```

```
# Для Raspberry Pi и других ARM-устройств
# ARMv7, ARMv8 (aarch64)
dpkg --print-architecture
```

```
# armhf, arm64
```

Другие архитектуры:

- PowerPC - серверные системы IBM
- RISC-V - открытая архитектура
- s390x - мейнфреймы IBM

Управление устройствами

```
bash
```

```
# Просмотр устройств
lspci          # PCI устройства
lsusb          # USB устройства
lsblk          # Блочные устройства (диски)
```

```
# Драйверы ядра
```

```
lsmod                                # Загруженные модули
```

```
modinfo module_name                # Информация о модуле
```



Уровень 2: Ядро Linux (Kernel Layer)

Монолитное ядро с модулями

Структура ядра:

```
bash
```

```
# Версия ядра
```

```
uname -r
```

```
# 5.15.0-91-generic
```

```
# Конфигурация ядра
```

```
zcat /proc/config.gz | grep CONFIG_
```

Основные подсистемы ядра

1. Управление процессами (Process Management)

```
bash
```

```
# Системные вызовы для работы с процессами
```

```
strace ls                            # Просмотр системных вызовов
```

```
# Процессы в /proc
```

```
ls /proc/                            # Виртуальная файловая система
```

```
cat /proc/1/status                   # Информация о процессе init
```

2. Управление памятью (Memory Management)

```
bash
```

```
# Информация о памяти
```

```
cat /proc/meminfo
```

```
free -h
```

```
# Виртуальная память
```

```
vmstat 1 5 # Статистика виртуальной памяти
```

3. Файловые системы (File Systems)

```
bash
```

```
# Поддерживаемые файловые системы
```

```
cat /proc/filesystems
```

```
# Монтирование файловых систем
```

```
mount | grep ^/dev
```

```
findmnt
```

4. Сетевой стек (Network Stack)

```
bash
```

```
# Сетевые интерфейсы
```

```
ip addr show
```

```
cat /proc/net/dev
```

5. Драйверы устройств (Device Drivers)

```
bash
```

```
# Драйверы в sysfs
ls /sys/class/net/      # Сетевые устройства
```

```
ls /sys/block/          # Блочные устройства
```

Модули ядра

```
bash
```

```
# Работа с модулями
sudo modprobe module_name  # Загрузка модуля
sudo rmmod module_name     # Выгрузка модуля
sudo depmod                # Обновление зависимостей
```

```
# Пример: загрузка драйвера USB
```

```
sudo modprobe usb-storage
```



Уровень 3: Системные вызовы (System Call Interface)

Что такое системные вызовы

```
// Пример системных вызовов в C
#include <unistd.h>
#include <sys/syscall.h>

// Открытие файла
int fd = open("file.txt", O_RDONLY);

// Чтение из файла
read(fd, buffer, size);
```

```
// Запись в файл
```

```
write(fd, buffer, size);
```

Основные категории системных вызовов

Управление процессами:

```
bash
```

```
# Просмотр системных вызовов процесса
```

```
strace -e trace=process command
```

```
# Основные вызовы:
```

```
# - fork()      - создание процесса
```

```
# - exec()      - выполнение программы
```

```
# - wait()      - ожидание завершения
```

```
# - exit()      - завершение процесса
```

Файловые операции:

```
bash
```

```
# Мониторинг файловых операций
```

```
strace -e trace=file command
```

```
# Основные вызовы:
```

```
# - open()      - открытие файла
```

```
# - read()      - чтение
```

```
# - write()     - запись
```

```
# - close()     - закрытие
```



Уровень 4: Системные библиотеки и утилиты

Стандартные библиотеки

GNU C Library (glibc):

```
bash
```

```
# Версия glibc
```

```
ldd --version
```

```
# Зависимости бинарного файла
```

```
ldd /bin/ls
```

```
# Основные функции:
```

```
# - malloc/free - управление памятью
```

```
# - printf      - вывод
```

```
# - fopen/fclose - работа с файлами
```

Другие важные библиотеки:

```
bash
```

```
# Просмотр используемых библиотек
```

```
lssof -p $$ | grep -E "\.so"
```

```
# Основные библиотеки:
```

```
# - libc.so.6      - основная C библиотека
```

```
# - libm.so.6      - математические функции
```

```
# - libpthread.so.0 - потоки POSIX
```

```
# - libdl.so.2     - динамическая загрузка
```

Системные утилиты

Coreutils - основные утилиты:

```
bash
```

```
# Файловые операции
```

```
cp, mv, rm, ls, mkdir
```

```
# Текстовые утилиты
```

```
cat, grep, sed, awk, sort
```

```
# Системная информация
```

```
ps, top, df, du, free
```

Системные демоны (daemons):

```
bash
```

```
# Просмотр демонов
```

```
systemctl list-units --type=service
```

```
# Ключевые демоны Ubuntu:
```

```
# - systemd      - менеджер системы и служб
```

```
# - NetworkManager - управление сетью
```

```
# - dbus          - межпроцессное взаимодействие
```

```
# - cron           - планировщик задач
```

Уровень 5: Пользовательские приложения

Типы приложений в Ubuntu

Графические приложения (GUI):

```
bash
```

```
# Рабочие столы в Ubuntu
```

```
echo $XDG_CURRENT_DESKTOP
```

```
# GNOME, KDE, XFCE, etc.
```

```
# Графические серверы
```

```
ps aux | grep -E "(Xorg|wayland)"
```

Консольные приложения (CLI):

```
bash
```

```
# Эмуляторы терминала
```

```
ps -p $$ -o comm=
```

```
# gnome-terminal, konsole, xterm, etc.
```

```
# Оболочки (Shells)
```

```
echo $SHELL
```

```
# bash, zsh, fish, etc.
```

Пакетная система Ubuntu

Архитектура пакетного менеджмента:

```
bash
```

```
# Формат пакетов DEB
```

```
dpkg -c package.deb      # Содержимое пакета
```

```
dpkg -I package.deb      # Информация о пакете
```

```
# Репозитории
```

```
cat /etc/apt/sources.list
```

```
ls /etc/apt/sources.list.d/
```

Уровни абстракции пакетов:

```
text
```

APT (apt)	Высокоуровневый менеджер
dpkg	Низкоуровневый менеджер .deb
.deb файлы	Архивы с файлами и метаданными

Архитектура загрузки системы

Процесс загрузки Ubuntu (systemd)

1. BIOS/UEFI инициализация:

```
bash
```

```
# Информация о BIOS
sudo dmidecode -t bios
```

```
# UEFI переменные
```

```
ls /sys/firmware/efi/efivars/
```

2. Загрузчик GRUB:

```
bash
```

```
# Конфигурация GRUB
cat /boot/grub/grub.cfg
```

```
sudo update-grub          # Обновление конфигурации
```

3. Инициализация ядра:

```
bash
```

```
# Параметры ядра при загрузке
```

```
cat /proc/cmdline
```

```
# Ранний userspace (initramfs)
```

```
lsinitramfs /boot/initrd.img-$(uname -r)
```

4. Systemd - процесс init:

```
bash
```

```
# Дерево процессов
```

```
systemctl --no-pager
```

```
pstree -p
```

```
# Анализ времени загрузки
```

```
systemd-analyze
```

```
systemd-analyze critical-chain
```



Подсистемы ядра Linux

Виртуальная файловая система (VFS)

Унифицированный интерфейс для файловых систем:

```
bash
```

```
# Поддерживаемые файловые системы
```

```
cat /proc/filesystems
```

```
# Монтирование различных ФС
```

```
mount -t ext4 /dev/sda1 /mnt
```

```
mount -t nfs server:/share /mnt
```

```
mount -t proc proc /proc
```

Сетевой стек

Уровни сетевого стека:

```
bash
```

```
# Сетевые интерфейсы
```

```
ip link show
```

```
# Таблица маршрутизации
```

```
ip route show
```

```
# Статистика
```

```
ss -tuln
```

```
# Сокеты
```

```
cat /proc/net/snmp
```

```
# Сетевая статистика
```

Подсистема безопасности

Модули безопасности:

```
bash
```

```
# AppArmor в Ubuntu
```

```
sudo aa-status
```

```
# Статус AppArmor
```

```
sudo apparmor_parser
```

```
# Парсер профилей
```

```
# SELinux (в некоторых дистрибутивах)
```

```
sestatus
```

```
getenforce
```



Мониторинг архитектуры системы

Анализ системных ресурсов

```
bash
```

```
# Процессы и ресурсы
```

```
htop # Интерактивный монитор
```

```
iotop # Мониторинг I/O
```

```
nethogs # Сетевой трафик по процессам
```

```
# Производительность ядра
```

```
perf stat ls # Статистика производительности
```

Отладочная информация

```
bash
```

```
# Отладочная информация системы
```

```
dmesg # Сообщения ядра
```

```
journalctl # Логи systemd
```

```
# Профилирование системы
```

```
strace -c command # Статистика системных вызовов
```

```
ltrace -c command # Статистика библиотечных вызовов
```



Диагностика архитектурных проблем

Анализ зависимостей

```
bash
```

```
# Анализ бинарных файлов
```

```
file /bin/ls
```

```
readelf -h /bin/ls
```

```
objdump -x /bin/ls
```

```
# Проверка библиотек
```

```
ldd /usr/bin/application
```

Отслеживание системных вызовов

```
bash
```

```
# Мониторинг в реальном времени
```

```
sudo strace -p PID # Отслеживание процесса
```

```
sudo perf trace ls # Современный трассировщик
```

```
# Анализ системных вызовов
```

```
strace -e open,read,write command
```

Эта архитектура обеспечивает стабильность, безопасность и гибкость Ubuntu, позволяя системе эффективно работать на различных аппаратных платформах и удовлетворять разнообразные потребности пользователей.

Глубокое руководство по синтаксису командной строки Linux



Архитектура командной оболочки

Процесс обработки команд в Bash

text

Ввод команды → Разделение на токены → Подстановки → Выполнение



Лексический анализ и токенизация

Токены и метасимволы

Разделители команд:

bash

Точка с запятой - последовательное выполнение

`ls; pwd; whoami`

Амперсанд - фоновое выполнение

`sleep 10 &`

Вертикальная черта - конвейер

`ls -l | grep "txt" | wc -l`

Операторы управления:

bash

Группировка команд

`{ ls -l; date; } > log.txt`

`( cd /tmp && ls ) # Подпроцесс`

Логические операторы

`true && echo "Успех" # AND`

`false || echo "Провал" # OR`

`! grep "error" file.log # NOT`



Подстановки (Expansions) - глубокий разбор

1. Brace Expansion (подстановка фигурными скобками)

Базовые варианты:

```
bash

# Последовательности
echo {1..5}          # 1 2 3 4 5
echo {a..e}          # a b c d e
echo {5..1}          # 5 4 3 2 1
echo {01..10}        # 01 02 03 ... 10

# Комбинации
echo {A,B,C}{1,2}    # A1 A2 B1 B2 C1 C2

mkdir /home/user/{docs,music,videos}/{2020,2021,2022}
```

Сложные примеры:

```
bash

# Вложенные подстановки
echo {A{1,2},B{3,4}} # A1 A2 B3 B4

# С шагом
echo {1..10..2}       # 1 3 5 7 9

echo {z..a..2}        # z x v t r p n l j h f d b
```

2. Tilde Expansion (подстановка тильды)

Расширенные варианты:

```
bash

echo ~                # Домашняя директория текущего пользователя
echo ~username        # Домашняя директория указанного пользователя
echo ~+               # Текущая директория (аналог $PWD)
echo ~-               # Предыдущая директория (аналог $OLDPWD)

# Использование в путях
ls ~/Documents

cp file.txt ~/backups/
```

3. Parameter Expansion (подстановка параметров)

Базовый синтаксис:

```
bash
name="John"
echo $name           # John

echo ${name}         # John - явный синтаксис
```

Расширенные операции со строками:

```
bash
filename="document.txt.backup"

# Длина строки
echo ${#filename}    # 20

# Подстроки
echo ${filename:0:8}  # document
echo ${filename:(-6)} # .backup (от конца)

# Замена
echo ${filename/.txt/.pdf}    # document.pdf.backup
echo ${filename//./_}        # document_txt_backup

# Удаление по шаблону
echo ${filename%.*}           # document.txt (удалить с конца)
echo ${filename%%.*}          # document (жадное удаление)
echo ${filename#*.}           # txt.backup (удалить с начала)

echo ${filename##*.}          # backup (жадное удаление)
```

Проверки и значения по умолчанию:

```
bash
unset var

# Использование значений по умолчанию
echo ${var:-default}        # default (var не установлена)
echo ${var:=default}        # default + присваивание var
echo ${var:?Error message}   # Ошибка если var не установлена

echo ${var:+replacement}    # replacement если var установлена
```

4. Command Substitution (подстановка команд)

Сравнение синтаксисов:

```
bash

# Современный синтаксис (рекомендуется)
files=$(ls)
count=$(wc -l < file.txt)

# Устаревший синтаксис
files=`ls`

count=`wc -l < file.txt`
```

Вложенные подстановки:

```
bash

# Подстановка внутри подстановки
current_dir=$(basename $(pwd))
file_count=$(ls $(pwd) | wc -l)

# Сложные примеры
archive_name="backup_$(date +%Y%m%d_%H%M%S).tar.gz"

process_list=$(ps aux | grep $(whoami) | awk '{print $2}')
```

5. Arithmetic Expansion (арифметические подстановки)

Различные формы:

```
bash

# Базовые операции
echo $(( 5 + 3 ))      # 8
echo $(( 10 / 3 ))     # 3 (целочисленное деление)
echo $(( 10 % 3 ))     # 1 (остаток)

# Разные системы счисления
echo $(( 0xFF ))       # 255 (шестнадцатеричное)
echo $(( 2#1010 ))     # 10 (двоичное)
echo $(( 8#77 ))       # 63 (восьмеричное)

# Инкременты и декременты
x=5
```

```
echo $(( x++ ))          # 5 (постинкремент)
```

```
echo $(( ++x ))          # 7 (преинкремент)
```

Битовые операции:

```
bash
```

```
echo $(( 5 & 3 ))         # 1 (AND)
```

```
echo $(( 5 | 3 ))         # 7 (OR)
```

```
echo $(( 5 ^ 3 ))         # 6 (XOR)
```

```
echo $(( ~5 ))            # -6 (NOT)
```

```
echo $(( 5 << 1 ))        # 10 (сдвиг влево)
```

```
echo $(( 5 >> 1 ))        # 2 (сдвиг вправо)
```

6. Process Substitution (подстановка процессов)

Сравнение с конвейерами:

```
bash
```

```
# Для команд, требующие файлы вместо stdin
```

```
diff <(ls dir1) <(ls dir2)
```

```
# Множественные входы
```

```
paste <(seq 1 5) <(seq 6 10) <(seq 11 15)
```

```
# Сохранение переменных в подпроцессах
```

```
var="value"
```

```
( var="new"; echo "Inside: $var" ) # Inside: new
```

```
echo "Outside: $var"             # Outside: value
```

Подстановка путей (Pathname Expansion)

Globbing patterns

Базовые шаблоны:

```
bash
```

```
ls *.txt                  # Все .txt файлы
```

```
ls ?.txt                  # Один символ: a.txt, 1.txt
```

```
ls [abc].txt          # a.txt, b.txt, или c.txt
ls [a-c].txt          # Диапазон: a.txt, b.txt, c.txt

ls [^a].txt           # Все кроме a.txt
```

Расширенные шаблоны (extglob):

bash

Включение расширенного globbing

```
shopt -s extglob
```

Сложные шаблоны

```
ls *(pattern)         # 0 или более повторений
ls +(pattern)         # 1 или более повторений
ls ?(pattern)         # 0 или 1 повторение
ls @(pat1|pat2)       # Один из шаблонов
ls !(pattern)         # Все кроме шаблона
```

Примеры

```
ls *.*(txt|pdf)       # Все .txt или .pdf файлы
```

```
ls !(*.bak)           # Все файлы кроме .bak
```

Рекурсивный globbing:

bash

Включение рекурсивного поиска

```
shopt -s globstar
```

Рекурсивные шаблоны

```
ls **/*.txt           # Все .txt файлы в поддиректориях
```

```
find **/*.java        # Рекурсивный поиск Java файлов
```



Квотирование (Quoting) - глубокий разбор

Уровни экранирования

1. Backslash (обратный слэш):

bash

Экранирование одного символа

```
echo \SPATH          # Выведет: $PATH
echo \"Hello\"        # Выведет: "Hello"
echo Line1\          # Перенос строки
> Line2              # Продолжение
```

```
# Специальные символы
echo -e "Tab\tSeparated"

echo -e "Line1\nLine2"
```

2. Single Quotes (одинарные кавычки):

```
bash

# Буквальная интерпретация
echo '$HOME && `date`' # $HOME && `date`

# Исключения - нельзя экранировать саму кавычку
# echo 'It\'s error'   # Синтаксическая ошибка

echo 'It\'\'s working' # It's working
```

3. Double Quotes (двойные кавычки):

```
bash

name="John"
echo "Hello $name"      # Hello John
echo "Today is $(date)" # Подстановка команд работает
echo "Home: $HOME"      # Подстановка переменных работает

# Экранирование в двойных кавычках
echo "Price: \$100"     # Price: $100

echo "Path: \"SPATH\"" # Path: "..."
```

ANSI-C Quoting ('\$string')

Специальные последовательности:

```
bash

echo '$Line1\nLine2' # Две строки
echo '$Tab\tseparated' # Табуляция
echo '$Bell: \a'      # Звонок (если поддерживается)
echo '$Backspace: \b' # Backspace
```

```
echo $'Escape: \e'      # Escape символ
```

```
echo $'Unicode: \u263A' # Смайлик ☺
```

Практическое использование:

```
bash
```

```
# Цветной вывод
```

```
RED=$'\033[31m'
```

```
RESET=$'\033[0m'
```

```
echo "${RED}Error${RESET}"
```

```
# Специальные символы в переменных
```

```
NL=$'\n'
```

```
TAB=$'\t'
```



Перенаправления ввода/вывода - расширенный синтаксис

Дескрипторы файлов

Стандартные дескрипторы:

```
bash
```

0 - STDIN (стандартный ввод)

1 - STDOUT (стандартный вывод)

2 - STDERR (стандартный вывод ошибок)

3-9 - Пользовательские дескрипторы

Расширенное перенаправление:

```
bash
```

```
# Явное указание дескрипторов
```

```
command 2>&1      # stderr в stdout
```

```
command 1>&2      # stdout в stderr
```

```
# Группировка перенаправлений
```

```
{ command1; command2; } > output.txt 2>&1
```

```
# Перенаправление для части команды
ls -l > list.txt      # Только для ls

(ls; pwd) > output.txt # Для группы команд
```

bash

```
# Базовый синтаксис
cat << EOF
Строка 1
Строка 2
Строка 3
EOF

# С подавлением подстановок
cat << 'EOF'
Текст $VAR не будет подставлен
EOF
```

```
# С отступами (Bash 4.0+)
```

```
cat <<- EOF
    Текст с отступами
    будет выровнен
```

EOF

bash

```
# Передача строки в stdin
grep "pattern" <<< "text with pattern"
bc <<< "2 + 2"
```

```
read first last <<< "John Doe"
```

Пользовательские дескрипторы

Создание и использование:

bash

```
# Открытие дескриптора для чтения
exec 3< input.txt
```

```
read line <&3
exec 3<&-          # Закрытие
```

Открытие дескриптора для записи

```
exec 4> output.txt
echo "Test" >&4
exec 4>&-
```

Двухнаправленные дескрипторы

```
exec 5<> file.txt
```

Управление выполнением команд

Приоритеты и группировка

Приоритет операторов:

```
bash

# Высший приоритет: |, ||, &&, ;
ls | grep txt && echo "Found" || echo "Not found"

# Изменение приоритета скобками
(ls | wc -l) && echo "Counted"

# Подробный пример
false || true && echo "Executed"    # Executed

false || (true && echo "Executed") # Executed
```

Группировка команд:

```
bash

# Группировка в подпроцессе (subshell)
( cd /tmp; ls )          # cd не влияет на текущую оболочку

# Группировка в текущей оболочке
{ cd /tmp; ls; }         # cd влияет на текущую оболочку

# Синтаксические различия
{ ls; pwd; }             # Пробелы обязательны!
```

```
(ls; pwd) # Без пробелов
```

Условное выполнение

Сложные условия:

```
bash
```

```
# Цепочки условий
```

```
command1 && command2 && command3
```

```
command1 || command2 || command3
```

```
# Комбинирование
```

```
command1 && command2 || command3
```

```
# Группировка в условиях
```

```
[ -f file.txt ] && { echo "Exists"; cp file.txt backup/; }
```

Prompt String (PS1) - кастомизация

Специальные escape-последовательности

Базовые элементы:

```
bash
```

```
# Текущая настройка
```

```
echo $PS1
```

```
# Пример кастомизации
```

```
PS1='\u@\h:\w$ ' # user@host:dir$
```

```
PS1='[\t] \u:\w$ ' # [time] user:dir$
```

```
# Цвета
```

```
PS1='\[\033[32m\]\u@\h\[\033[0m\]:\[\033[34m\]\w\[\033[0m\]$ '
```

Доступные escape-коды:

```
bash
```

```
\d - Дата (День недели Месяц Дата)
```

\H - Полное имя хоста
\h - Короткое имя хоста
\t - Время HH:MM:SS (24-часовой формат)
\T - Время HH:MM:SS (12-часовой формат)
\@ - Время в AM/PM формате
\u - Имя текущего пользователя
\w - Текущая рабочая директория
\W - Базовая имя текущей директории
\! - Номер команды в истории

\# - Номер команды в текущей сессии

Отладка и анализ синтаксиса

Включение отладочного режима

Опции отладки:

```
bash
```

```
#!/bin/bash
```

```
# Подробный вывод
```

```
set -v          # Echo commands
```

```
set -x          # Trace commands
```

```
# Строгий режим
```

```
set -e          # Exit on error
```

```
set -u          # Exit on undefined variable
```

```
set -o pipefail # Exit on pipe failure
```

```
# Комбинирование
```

```
set -euxo pipefail # Строгий режим
```

Анализ синтаксиса:

```
bash
```

```
# Проверка синтаксиса без выполнения
```

```
bash -n script.sh # Syntax check
```

```
bash -v script.sh # Verbose mode
```

```
bash -x script.sh # Trace mode
```

```
# Пошаговое выполнение
```

```
trap 'read -p "Line: $LINENO"' DEBUG
```

Визуализация обработки команд

Процесс разбора:

```
bash
```

```
# Пример сложной команды для анализа
```

```
archive_name="backup_$(date +%F).tar.gz"
find /home -name "*.doc" -o -name "*.pdf" | \
    xargs tar -czf "${archive_name}" 2>/dev/null && \
```

```
    echo "Backup completed" || echo "Backup failed"
```

Этапы обработки:

1. Токенизация: Разделение на команды, операторы
2. Подстановки: Переменные, команды, арифметика
3. Разделение слов: По пробелам и разделителям
4. Подстановка путей: Globbing шаблоны
5. Выполнение: Запуск команд

Этот глубокий разбор синтаксиса позволяет понять, как оболочка обрабатывает команды на каждом этапе, что необходимо для написания сложных скриптов и эффективной работы в командной строке.